Prepared by Sai Veda

LLM Observability & Eval

Author	Document date	Validation
Sai Veda	June 3, 2025	21 automated tests

Abstract

This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools.

Project Background

At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack.

Headline Result

5M traces / 1.79B tokens; PSI flags injected drift

Scope Overview

Synthetic trace generation with tenants, models, and time windows. Partitioned Parquet lake with DuckDB analytics. PSI or KL drift signals plus heuristic and logistic quality checks. Dashboards and scorecards for model and fleet review.

Project Context

Trace analytics + quality monitoring for LLM fleets at scale - a self-contained, fully offline reference implementation. It generates a realistic multi-tenant LLM trace firehose, lands it as partitioned Parquet, and runs four production concerns over it: cost/latency analytics (DuckDB, out-of-core), embedding-drift detection (PSI/KL), a hallucination detector (heuristic + logistic), and a per-model eval scorecard - with generated, product-grade dashboards.

Executive Summary

Background, objectives, scope, and project impact.

Project Background

At production volume, trace data becomes operational exhaust unless it is modeled properly. Teams need to understand spend, latency, model quality, and distribution change without moving to an online-only stack.

Executive Summary

This brief documents an observability stack for LLM fleets where trace analytics, cost monitoring, drift detection, and hallucination checks share the same data backbone instead of living in disconnected tools.

Objectives

- Generate and store a realistic multi-tenant trace firehose.
- Answer cost, latency, and token questions out of core.
- Detect distribution shift and likely hallucinations early.

Scope

- Synthetic trace generation with tenants, models, and time windows.
- Partitioned Parquet lake with DuckDB analytics.
- PSI or KL drift signals plus heuristic and logistic quality checks.
- Dashboards and scorecards for

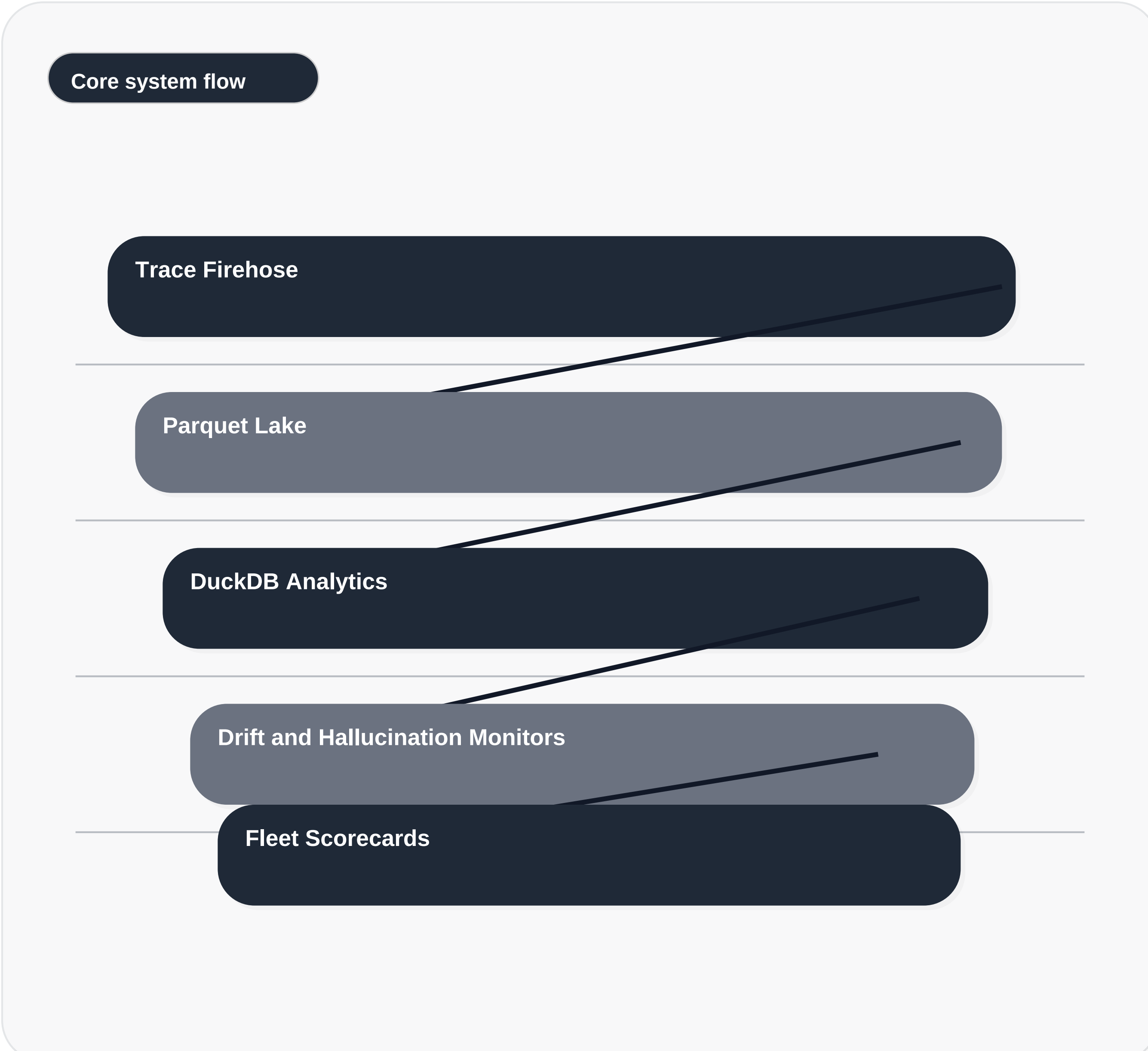
Impact

The project demonstrates that a local, partitioned analytics path can still support fleet-level insight. The same trace lake supports operational reporting, shift detection, and model-quality scorecards in one review flow.

5M traces / 1.79B tokens; PSI flags injected drift

Solution Architecture

Trace storage, analytics, and quality detectors stay modular so operations and model-risk teams can inspect the same pipeline.



Design Note 1

Operational analytics and model-risk analytics share one trace backbone to avoid fragmented reporting.

Design Note 2

Drift is measured before customer-visible breakage, which makes the system proactive rather than reactive.

Design Note 3

Out-of-core SQL was preferred so the story stays reproducible on a single machine.

Implementation Process

Delivery phases, engineering approach, and quality checkpoints.

Delivery Narrative

Trace analytics + quality monitoring for LLM fleets at scale - a self-contained, fully offline reference implementation. It generates a realistic multi-tenant LLM trace firehose, lands it as partitioned Parquet, and runs four production concerns over it: cost/latency analytics (DuckDB, out-of-core), embedding-drift detection (PSI/KL), a hallucination detector (heuristic + logistic), and a per-model eval scorecard - with generated, product-grade dashboards.

1 Trace model

Defined the schema around cost, tokens, latency, model identity, tenant, and quality labels.

2 Storage path

Wrote traces into partitioned Parquet so analytics could scale without loading everything into memory.

3 Quality monitoring

Layered drift signals and hallucination flags on top of the same trace stream.

4 Review surface

Rendered dashboards that connect fleet metrics to model-level quality scorecards.

Quality Checkpoints

- 21 automated tests validate core behavior and edge cases.
- Benchmark files and screenshots are generated from committed project artifacts.
- The run path stays deterministic so numbers can be reproduced and reviewed directly.

Engineering Principles

- Operational analytics and model-risk analytics share one trace backbone to avoid fragmented reporting.
- Drift is measured before customer-visible breakage, which makes the system proactive rather than reactive.
- Out-of-core SQL was preferred so the story stays reproducible on a single machine.

Evidence And Results

Test evidence, benchmark interpretation, and screenshot review.

21 passing tests | **June 3, 2025** | **5M traces / 1.79B tokens; PSI flags injected drift**

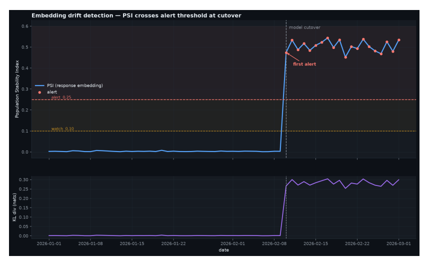
Benchmark Evidence

This repository emphasizes visual benchmark artifacts and automated validation outputs. Where tables are not present, the screenshots and test fixtures remain the primary review surface.

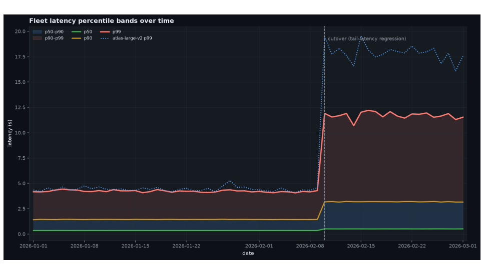
Result Interpretation

- The evidence page shows that large trace volumes can still be queried quickly when storage is partitioned correctly.
- PSI highlights injected drift cleanly, which gives the quality layer an early warning signal.
- The visual scorecards make per-model trade-offs legible for both platform and risk

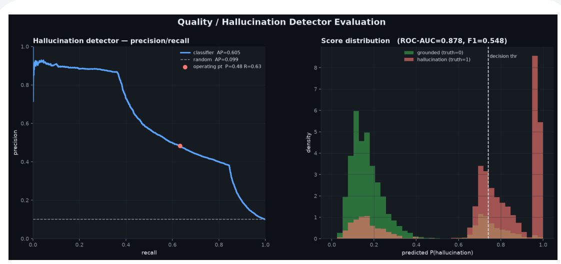
Fleet dashboard - KPI tiles, latency-over-time, spend/tokens by model



Drift detection - PSI crosses the alert threshold exactly at the model cutover



Latency percentile bands - the injected tail-latency regression is unmistakable



Hallucination detector - PR curve + score distribution vs a random baseline

Risks And Next Steps

Operational considerations, mitigation choices, and forward path.

Risk Register

Signal fatigue	Handled by separating operational metrics from model-quality indicators and showing both on one timeline.
Storage sprawl	Kept under control through partitioned Parquet and analytic query pushdown.
Weak quality proxies	Balanced by combining heuristic flags with a learned hallucination detector.

Next Steps

- Add human-review annotations and slice-aware eval routing by tenant or product workflow.
- Introduce alert policies that combine drift, cost, and quality instead of triggering on single metrics.
- Extend the dashboard into a comparative release-review workflow for prompt or model cutovers.

Closing Note

The project brief is intentionally written as delivered work rather than a transfer note. It documents the business problem, the engineering choices, the measured evidence, and the remaining operational path in a format suitable for portfolio review.

Sai Veda

Project brief date: June 3, 2025

5M traces / 1.79B tokens; PSI flags injected drift